BUILD YOUR SETUP

PC Configuration & Pricing Application

Final Project Report — Software Design Patterns

Prepared from the submitted source code, project ZIP and course rubric

November 2026

1. Introduction

Build Your Setup is a Java-based PC configurator with a web graphical interface. It allows a user to assemble a computer from catalog components, configure optional upgrades, select a pricing strategy, inspect compatibility, place/cancel an order and export a build report.

The project addresses the course idea “Build Your Setup”: configure a gaming or work PC, check compatibility and calculate the price. The implementation uses Java 17+ and a lightweight HTML/CSS/JavaScript front end; no external Java libraries are required.

2. Problem Statement and Target Users

Problem. Selecting PC components can be confusing: parts must fit together and the total price changes when optional services or discounts are applied. The application centralizes component selection, compatibility checks and price calculation.

Target users. People planning a custom PC, students comparing configurations, and users who want to save or order a build.

3. User Scenarios

#	Scenario	User outcome
1	Create a build	User enters a name; a new configuration is created and saved.
2	Configure components	User selects CPU, motherboard, RAM, GPU, storage and PSU; the summary updates.
3	Check compatibility	Socket and power rules are evaluated; issues are shown and logged.
4	Add upgrades / switch pricing	User toggles assembly/RGB/overclock/warranty and chooses Standard, Student or Volume pricing.
5	Save/export and order	A compatible complete build can be exported as a text report and placed as an order; an order can later be cancelled.

Core CRUD actions include creating, viewing, renaming, configuring and deleting builds. Orders support placement, viewing and cancellation. The final output is a downloadable PC build report.

4. Functional Requirements Coverage

Requirement	Evidence in submitted project
3+ related domain entities	Component, PcBuild and Order (plus catalog/price/event types).
Interchangeable behavior	PricingStrategy: Standard, Student and Volume discount algorithms selectable at runtime.
Event-driven reaction	BuildEventBus publishes build changes; ActivityLog and CompatibilityWatcher respond.
Persistence	BuildRepository and OrderRepository store/reload data in files under data/.
Validation/error handling	ValidationException guards names, IDs, prices, unknown options and incomplete/invalid operations.
Final output and sample data	Text report export; SetupFacade seeds sample builds when repository is empty.

5. System Architecture

The implementation separates concerns into layers. The browser UI communicates with the Java HTTP server using JSON/form requests. The server delegates business operations to SetupFacade. Domain rules, price calculation, event handling and repositories are kept in separate packages.

Layer/package	Responsibility
web (WebServer.java; web/index.html)	HTTP routes, UI rendering, forms, API calls and user feedback.
app (SetupFacade)	Facade entry point; orchestrates domain, repository, pricing and events.
domain	Component, PcBuild, Order, CompatibilityChecker, ComponentFactory and validation.
pricing	PriceItem abstraction, decorators, PriceCalculator and pricing strategies.
events	BuildEvent, BuildEventBus, listeners, ActivityLog and CompatibilityWatcher.
storage	Catalog, BuildRepository and OrderRepository; file persistence.

This separation supports focused methods, readable code and reduced coupling between UI, business logic and storage.

6. Design Patterns (6)

Pattern / type	Problem solved & participants	How to demonstrate
Builder — Creational	PcBuild.Builder constructs a configuration step-by-step and validates it at build(); toBuilder() supports edits while PcBuild remains immutable.	Create a build, add selected parts, then call build(); invalid blank name is rejected.
Factory Method — Creational	ComponentFactory.create() validates common inputs and delegates subtype creation to make(); concrete factories create CPU, motherboard, RAM, GPU, storage and PSU components.	Select/create component types from Catalog; each factory returns its corresponding Component subtype.
Decorator — Structural	PriceItem and Upgrades wrap a base price with optional Assembly, RGB, Overclock and Warranty additions without changing the base item.	Toggle upgrades; inspect price and report lines. Warranty is applied over prior subtotal.
Facade — Structural	SetupFacade offers a simplified API while hiding Catalog, repositories, checker, calculator and event bus from WebServer.	UI creates/configures a build through API endpoints; server delegates to facade.
Strategy — Behavioral	PricingStrategy encapsulates Standard, Student (10%) and Volume (12% from $2,000) discount algorithms; selection can change at runtime.	Switch strategy on a build and compare total prices.
Observer — Behavioral	BuildEventBus publishes events to BuildListener subscribers; ActivityLog and CompatibilityWatcher react independently.	Change CPU/board or other data; activity log updates and compatibility warning is generated.

Integration. All six patterns are used in the same build workflow: Factory-created components are assembled by Builder; Facade exposes operations; pricing combines Decorator and Strategy; events are published to Observer listeners.

7. UML Diagrams

The following diagrams are included in the project documentation and show the pattern participants and one principal interaction.

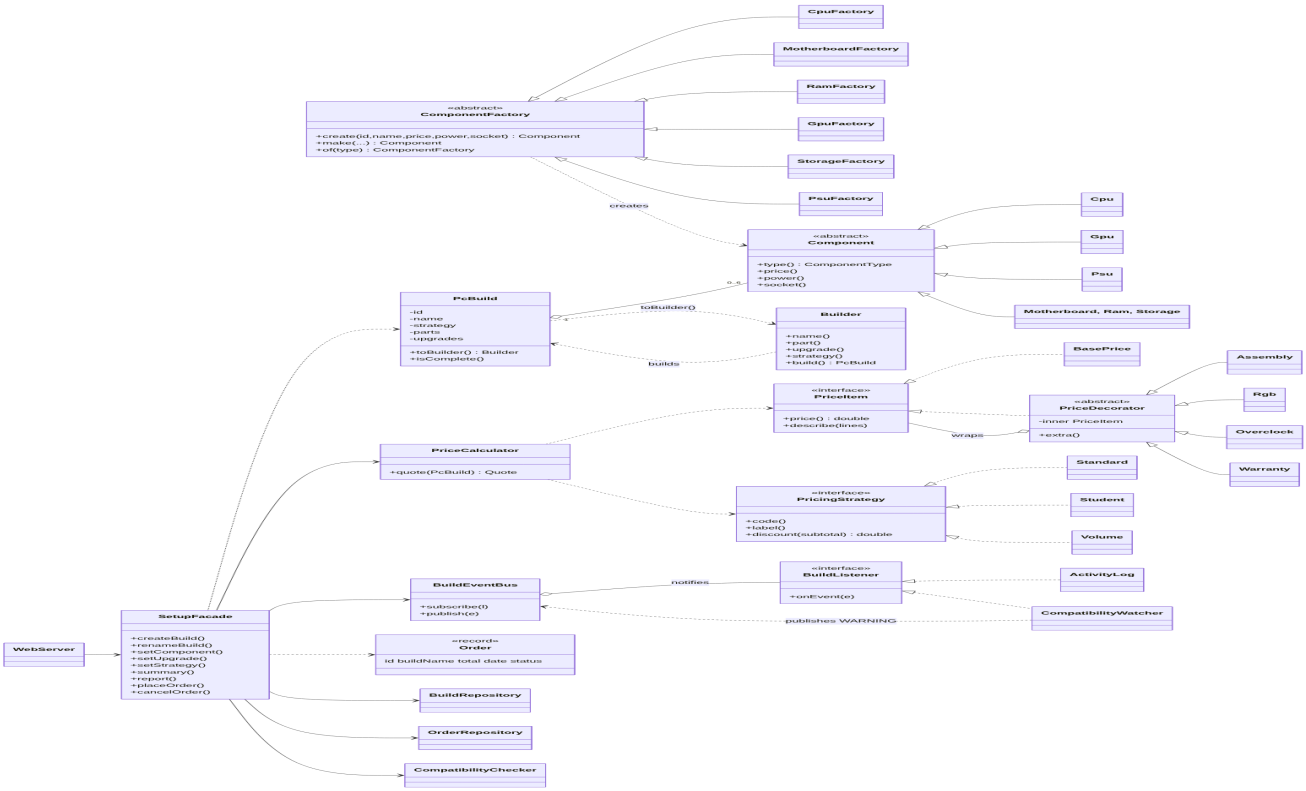


Figure 1. Class diagram (provided in project docs).

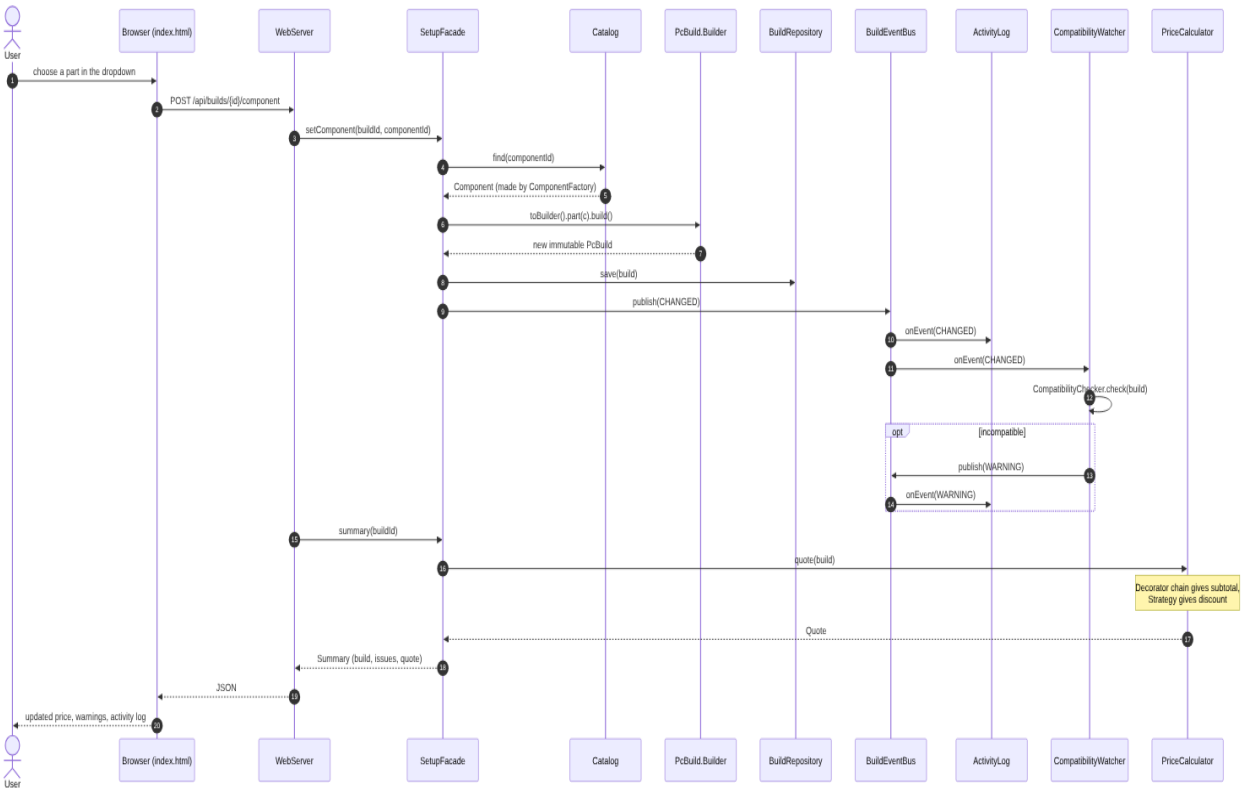


Figure 2. Sequence diagram (provided in project docs).

8. Application Interface

The interface has three main areas: saved builds on the left, configuration controls in the center, and build summary/orders/activity on the right. The screenshot below is a recreated illustrative view of the submitted HTML layout populated with representative Gaming Rig sample data; it is intended for report presentation and is not a raw browser capture.

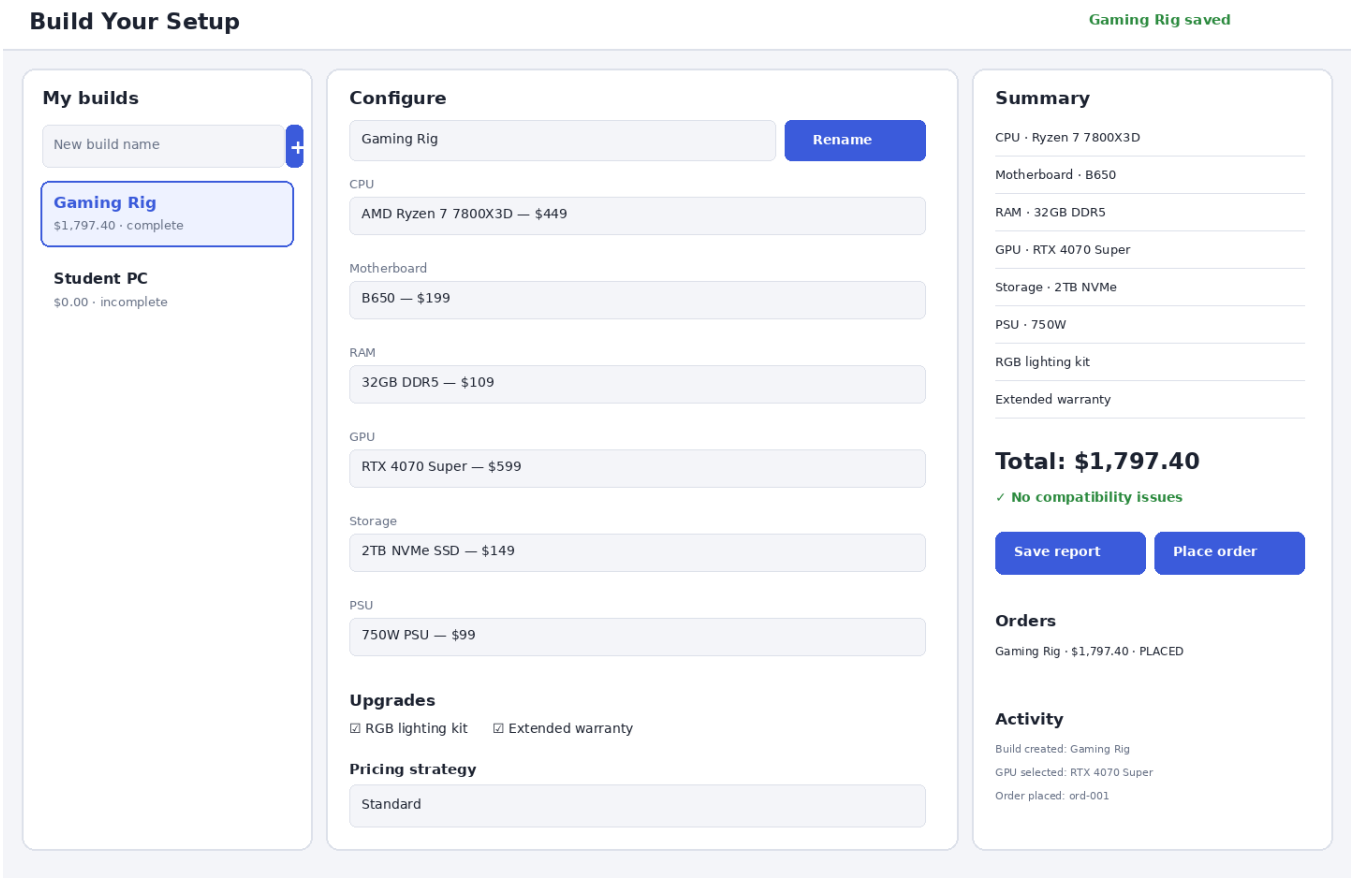


Figure 3. Build configuration interface — illustrative populated view.

9. Testing and Results

The project contains 10 automated scenarios in test/setup/TestRunner.java and a detailed test matrix in docs/TEST_SCENARIOS.md. On the recorded run (11 Nov 2026, Java 21), 10/10 scenarios passed.

Test	Coverage	Result
1	Main journey + Decorator total	PASS
2	Student/Volume Strategy switching	PASS
3	Observer compatibility event	PASS
4	Weak PSU boundary	PASS
5	Save and reload build	PASS
6	Invalid build names	PASS
7	Unknown component/upgrade/strategy/build IDs	PASS
8	Report export and incomplete-build rejection	PASS
9	Rename success and invalid names	PASS
10	Order placement, persistence and cancellation	PASS

Manual/API smoke checks recorded in the project: rename a build; place an order for the seeded Gaming Rig; cancel the order. The report should preserve both automated and manual evidence; the automated suite does not replace checking the complete browser journey on the defense computer.

10. Individual Contributions

The repository README proposes the following division. Confirm the actual team member names and that the assignments reflect each person's own work before submission:

Member (fill name)	Patterns / contribution
Student A	Factory Method; Builder
Student B	Decorator; Strategy
Student C	Observer; Facade; tests

The rubric requires each student to independently implement at least two patterns and explain their participants, interactions, benefits and limitations. For a team of two, redistribute all six patterns (three per person); for four members, document each person's contribution clearly and check the minimum pattern requirement with the instructor.

11. Reproducible Setup

Prerequisites: Java 17 or newer. Run commands from the project root:

```
javac -d out $(find src -name "*.java")
java -cp out setup.web.WebServer
# open http://localhost:8080

javac -d out $(find src test -name "*.java")
java -cp out setup.TestRunner
```

Data files are created automatically in data/; sample builds are seeded when the build repository is empty. No external Java dependencies are needed. Include source ZIP, repository link and this PDF in the Moodle submission.

12. Conclusion

Build Your Setup delivers a complete PC configuration workflow with component selection, compatibility validation, optional upgrades, configurable pricing, persistence, events, order lifecycle and report export. Its six design patterns are integrated into the working application rather than isolated examples. The automated suite provides evidence for the main logic and boundary behavior.

13. Future Work

Potential extensions include richer component filtering, additional compatibility rules (RAM generation, physical dimensions and connectors), price comparison across vendors, PDF/CSV export, user accounts and stronger end-to-end UI automation. These are future ideas and are not claimed as implemented.

Appendix A. Selected Code Excerpts

Builder (PcBuild.java):

```
public PcBuild build() {
if (name.isBlank()) throw new ValidationException("Build name must not be empty");
...
return new PcBuild(this);
}
```

Factory Method (ComponentFactory.java):

```
protected abstract Component make(String id, String name, double price, int power, String socket);
// concrete factories override make() for each component subtype
```

Decorator (Upgrades.java):

```
public double price() { return inner.price() + extra(); }
public void describe(List<String> lines) { inner.describe(lines); lines.add(label() + ...); }
```

Strategy (PricingStrategy.java):

```
public interface PricingStrategy { String code(); String label(); double discount(double subtotal); }
```

Observer (BuildEventBus.java):

```
public void publish(BuildEvent e) { for (BuildListener l : listeners) l.onEvent(e); }
```

Facade (SetupFacade.java): SetupFacade.summary(id) obtains the build, runs compatibility checks and calculates a price quote; WebServer uses this facade rather than directly coordinating domain services.

AI-use disclosure: The source README contains a placeholder statement naming Claude as used for the first code version. The team must revise this statement to truthfully list all AI tools and the actual scope of assistance, including ChatGPT if applicable.